SmartUzhavan

கிராம கணக்கு மேலாண்மை அமைப்பு

VERSION 3.1 — IMPLEMENTATION PLAN DOCUMENT

Feature Expansion | Module Restructure | Logic Redesign | PDF Reports

Project	SmartUzhavan — Rural Ledger & Machinery System
Version	3.1 — Feature Expansion & Restructure
Based On	SmartUzhavan_v3_Documentation.docx (Gold Master)
Total Changes	**12 Changes	8 Files Modified	3 Files New**
Date	April 2026

This document is the authoritative implementation plan for v3.1.
The Anti-Gravity Agent must read this document completely before writing any code.

1. Version 3.1 Overview

Version 3.1 is a feature expansion release built on top of the v3.0 Gold Master. The v3.0 system stabilized all crash-level issues, fixed core calculations, and established the backend PDF server. Version 3.1 expands the system's real-world usability by making financial records more flexible, reports more useful, the UI more consistent, and the module structure more logical.

All 12 changes in this version come directly from real-world usage feedback. They are mapped precisely to the existing codebase — this is not a redesign, it is a targeted expansion.

1.1 Complete Change Index

#	Module	Change	Files Affected	Type
C 01	Driver — Dashboard	Season+Year & Month+Year filter	DriverDashboard.jsx	Modify
C 02	Driver — Entry	Fix empty session time defaults	DriverEntry.jsx	Modify
C 03	Driver — Salary	Date range filter + PDF export	SalaryDriver→Drivers.jsx	Modify
C 04	Admin — Settings	Change any driver's login PIN	System.jsx (Settings tab)	Modify
C 05	Admin — Reports	PDF export for all 5 report types	System.jsx + server	Modify + Backend
C 06	Admin — Expenses	Fix undefined labels + 3 source-specific category lists	Expenses.jsx	Modify
C 07	Admin — Own Farm	Paddy vs Vaikool dynamic income form	OwnFarmIncome.jsx	Modify
C 08	Admin — Drivers	Two sub-modules + salary logic decouple from logs	Drivers.jsx + App.jsx	Major Expand
C 09	Admin — Workers	Two sub-modules + multi-worker entry + PDF reports	Workers.jsx + server	Major Expand
C 10	Admin — Navigation	Merge Config+Settings+Reports → System module	System.jsx + App.jsx	New File + Modify
C 11	Admin — Driver Logs	Admin edit/delete of driver log entries with cascade guard	DriverEntry.jsx	Modify
C 12	Admin — Worker Salary	Multi-worker selection for same work on same day	Workers.jsx	Modify

1.2 Files Summary

File	Action	Changes In This Version
src/pages/	Modify	Add season/month filter state and filtered stats

DriverDashboard.jsx		computation
src/pages/DriverEntry.jsx	Modify	Empty session defaults + Admin edit/delete log with cascade guard
src/pages/Drivers.jsx	Major Expand	Add 2 tabs: Drivers sub-module + Salary sub-module (absorbs SalaryDriver)
src/pages/SalaryDriver.jsx	Delete	Content moved into Drivers.jsx Tab 2 with new salary logic
src/pages/Expenses.jsx	Modify	Add en field to all categories + 3 source-specific category maps
src/pages/ OwnFarmIncome.jsx	Modify	Replace free-text crop with Paddy/Vaikool dynamic form
src/pages/Workers.jsx	Major Expand	Add 2 tabs, multi-worker entry, workerName→workerId, filters, PDF
src/pages/System.jsx	NEW	Combined module: Price Settings + System Settings + Reports
src/App.jsx	Modify	Remove 3 routes, add /system, update /drivers, remove /salary-driver
server/index.js	Modify	Add 9 new PDF endpoints (reports + worker + driver salary)
server/pdfGenerator.js	Modify	Add generateDriverSalaryPDF, generateWorkerReport, 5 report generators
src/services/storage.js	No change	updateRecord and deleteRecord already exist and are correct

2. Driver Module Changes

C01	Driver Dashboard	Add Season+Year and Month+Year filters

Current State (from code)

DriverDashboard.jsx has no filter state. All stats are computed by iterating every log and salary record in the collections unconditionally. The result is a single set of lifetime totals that grows forever and never reflects a specific period.

Why This Matters for Rural Users

Agriculture in Tamil Nadu runs on seasons — Kuruvai, Samba, Thaladi. A driver wants to know 'How much did I earn in the Samba 2025 season?' or 'How many hours did I work last month?' Lifetime totals answer neither question.

New Logic — Two Filter Modes

Add a filter toggle at the top of DriverDashboard.jsx with two modes:

Mode A — By Season (பருவம் வாரியாக)

- Season dropdown: KUR (குறுவை) | SAM (சம்பா) | THA (தலடி)
- Year input: 4-digit number
- Date range per season:
  - KUR = June 1 to September 30 of selected year
  - SAM = August 1 of selected year to January 31 of selected year+1
  - THA = October 1 of selected year to January 31 of selected year+1
- Filter logs: log.date falls within the season date range AND log.driverId === userId
- Filter salary: salary.date falls within the season date range AND salary.driverId === userId

Mode B — By Month (மாதம் வாரியாக)

- Month dropdown: January to December
- Year input: 4-digit number
- Filter logs: new Date(log.date).getMonth()+1 === selectedMonth && getFullYear() === selectedYear
- Filter salary: same logic on salary.date

Computed Stats (same cards, filtered data)

Stat Card	Computation	Tamil Label
Total Hours	filteredLogs.reduce((sum,l)=>sum+l.totalHours,0)	மொத்த மணி நேரம்

Days Worked	new Set(filteredLogs.map(l=>l.date)).size	வேலை நாட்கள்
Total Earned	filteredSalary.reduce((sum,s)=>sum+s.netPay,0)	சம்பாதித்தது
Total Received	filteredSalary.reduce sumPayments across all payments[]	பெற்றது
Pending Salary	Total Earned minus Total Received	நிலுவை சம்பளம்

Default State

On first load: Mode B (By Month), current month and year. Driver sees their current month's stats immediately without any interaction.

C02	Driver Entry	Fix session time defaults — empty inputs, correct trigger

Root Cause in Code

DriverEntry.jsx initialises sessions as:

```
sessions: [{ start: '09:00', end: '10:00', durationHours: 1 }]
```

This means every new log entry already has 1 hour pre-calculated. A driver who does not notice the pre-filled times and saves immediately will create an incorrect log entry. After a save, the reset also restores the same pre-filled defaults. There is no indication to the driver that they must change these values.

The Calculation Trigger (How It Works Now)

The handleSessionChange function checks: if (start && end) → calculate getDuration(). This logic is correct — it fires whenever either field changes as long as the other is non-empty. The only problem is the pre-filled values bypass this trigger visually.

Fix — Two Lines Changed

```
// BEFORE — in initial useState:
sessions: [{ start: '09:00', end: '10:00', durationHours: 1 }]

// AFTER — initial state:
sessions: [{ start: '', end: '', durationHours: 0 }]

// AFTER — reset after save:
sessions: [{ start: '', end: '', durationHours: 0 }]

// AFTER — addSession handler:
setSessions(prev => [...prev, { start: '', end: '', durationHours: 0 }])
```

With empty defaults, the duration shows 0.00 until the driver fills in both times. Once start and end are both non-empty, getDuration() fires and the result appears immediately without needing to touch any field a second time.

C03	Driver Salary	Date range filter + PDF export of salary history

Context — What Changes After C08

C03 describes additions to the Salary sub-module within Drivers.jsx (Tab 2). The standalone SalaryDriver.jsx is deleted. All features described here are implemented inside the new Drivers.jsx Salary tab. See C08 for the full salary logic redesign.

Date Range Filter

- Add two date inputs: From Date (தொடக்க தேதி) and To Date (முடிவு தேதி)
- Both default to empty (show all records)
- Filter applied: salary.date >= fromDate && salary.date <= toDate
- Clear Filter button resets both dates to empty
- Filtered count shown: 'Showing 5 of 12 entries'
- Driver logged in: can view their own filtered records in read-only mode
- Admin logged in: can view any driver's filtered records

PDF Export of Salary Statement

- 'Download Statement PDF' button appears below the filter controls
- Sends filtered salary records + driver info to backend endpoint
- Backend endpoint: POST /api/pdf/driver-salary-statement
- PDF layout: Driver name and village at top, period shown (from date to to date or 'All Records')
- Table rows: Date | Base | Bonus | Extra | Advance | Net Pay | Paid | Balance
- Footer: Totals row + Grand Total Outstanding
- Tamil headers on all columns
- PDF filename: driver-name-salary-YYYY-MM.pdf

3. Admin Module Changes

C04	System Settings	Change any driver's login PIN from admin settings

Location

This feature lives inside the new System.jsx module → System Settings (கணினி அமைப்பு) tab, in a 'Driver PIN Management' section below the Admin PIN change section.

Current State

Admin PIN change was added in v3.0 and stored in localStorage('admin_pin'). But driver PINs can only be changed by editing the driver record inside the Drivers module — there is no dedicated PIN management interface and no way to see that a PIN exists without editing.

New Logic

- Section header: 'Driver Login PIN (ஓட்டுநர் கடவுச்சொல்)'
- Driver dropdown: select from all active drivers in rl_drivers
- Current PIN display: masked as '****' (not revealed, only replaceable)
- New PIN input: 4-digit numeric input with validation
- Save button: calls updateRecord('rl_drivers', driverId, { pin: newPin })
- Success message in Tamil: 'PIN வெற்றிகரமாக மாற்றப்பட்டது'
- Validation: PIN must be exactly 4 digits and numeric — inline Tamil error if not

Security Behaviour

- The new PIN takes effect immediately on next driver login — no restart needed
- Old PIN is overwritten — there is no PIN history
- Admin can set all drivers to unique PINs or reset forgotten PINs

C05	Reports	PDF export button for each of the 5 report types

Current State

Reports.jsx renders 5 report types as HTML tables. There is no PDF generation capability at all. The reports cannot be printed, shared, or saved in any structured format.

Location

Reports live inside the new System.jsx module → Reports (அறிக்கைகள்) tab. The PDF buttons are added to each report's section.

5 Report PDF Endpoints — Logic Per Report

Report	Endpoint	Data Sent	PDF Contents
Monthly Summary	POST /api/pdf/report/monthly	{ month, year, incomeRows, expenseRows, totals }	Header: Month+Year. Two tables: Income by source, Expense by source. Bottom row: Net Profit
Machine-wise	POST /api/pdf/report/machine	{ season, year, machineRows }	Header: Season+Year. Table: Machine type, Job count, Total hours, Revenue, Expense, Profit
Farmer Dues	POST /api/pdf/report/farmer-dues	{ farmerRows }	Header: 'Outstanding Balances as of DATE'. Table: Farmer name, Village, Harvester due, Rental due, Total due. Sorted by total due descending
Driver Payroll	POST /api/pdf/report/driver-payroll	{ month, year, driverRows }	Header: Month+Year. Table: Driver name, Hours worked, Earned, Paid, Balance
Seasonal	POST /api/pdf/report/seasonal	{ season, year, jobRows, totals }	Header: Season+Year. Table: Bill ID, Farmer, Hours, Amount, Paid, Balance. Footer: Season totals

Frontend Behaviour

- Each report section has a 'Download PDF (PDF பதிவிறக்கம்)' button
- Button is disabled and greyed out until report data has been generated (filter applied and rendered)
- Button enabled only when the report table has at least one row
- On tap: sends current report data to the endpoint, receives PDF blob, auto-downloads
- PDF filename includes report type and period: monthly-summary-2026-04.pdf

C06	Expenses	Fix undefined labels + 3 source-specific category lists

Root Cause — Why 'undefined' Appears

Every category object in Expenses.jsx is structured as { value, ta } with no en field. The SelectField component renders labels as:

```
opt.label || `${opt.en} (${opt.ta})`
// opt.en is undefined for every category
// Result: 'undefined (இயந்திர பராமரிப்பு)'
```

This affects every category dropdown on the expenses screen. All English labels show as 'undefined'.

Root Cause — Why Same Categories for All Sources

A single CATEGORIES array is used regardless of the selected source. Maintenance and Spare Parts appear under Home Expenses. Food appears under Business. There is no semantic connection between source and categories.

Fix — Add en Field + 3 Separate Category Maps

```
// BEFORE (broken):
const CATEGORIES = [
  { value: 'maintenance', ta: 'இயந்திர பராமரிப்பு' },  // no en field
  ...
];

// AFTER (correct):
const CATEGORY_MAP = {
  business: [
    { value: 'maintenance',   en: 'Maintenance',    ta: 'இயந்திர பராமரிப்பு' },
    { value: 'diesel',        en: 'Fuel / Diesel',  ta: 'எரிபொருள் / டீசல்' },
    { value: 'parts',         en: 'Spare Parts',    ta: 'உதிரி பாகங்கள்' },
    { value: 'mechanic',      en: 'Mechanic Cost',  ta: 'மெக்கானிக் கட்டணம்' },
    { value: 'driver_food',   en: 'Driver Food',    ta: 'ஓட்டுநர் உணவு' },
    { value: 'tax',           en: 'Tax / Insurance',ta: 'வரி / காப்பீடு' },
    { value: 'other',         en: 'Other',          ta: 'இதர' },
  ],
  own_farm: [
    { value: 'seeds',         en: 'Seeds',          ta: 'விதைகள்' },
    { value: 'fertilizer',    en: 'Fertilizer',     ta: 'உரம்' },
    { value: 'pesticide',     en: 'Pesticide',      ta: 'பூச்சிக்கொல்லி' },
    { value: 'irrigation',    en: 'Irrigation',     ta: 'நீர்ப்பாசனம்' },
    { value: 'labour',        en: 'Labour',         ta: 'கூலி' },
    { value: 'equip_rental',  en: 'Equipment Rental',ta:'உபகரண வாடகை' },
    { value: 'other',         en: 'Other',          ta: 'இதர' },
  ],
  home_expense: [
    { value: 'food',          en: 'Food',           ta: 'உணவு' },
    { value: 'medical',       en: 'Medical',        ta: 'மருத்துவம்' },
    { value: 'education',     en: 'Education',      ta: 'கல்வி' },
    { value: 'transport',     en: 'Transport',      ta: 'போக்குவரத்து' },
    { value: 'utilities',     en: 'Utilities',      ta: 'மின்சாரம்/தண்ணீர்' },
    { value: 'other',         en: 'Other',          ta: 'இதர' },
  ],
};

// When source changes, reset category to first option of new source:
const handleSourceChange = (newSource) => {
  setFormData({...formData,
    source: newSource,
    category: CATEGORY_MAP[newSource][0].value
  });
};
```

C07	Own Farm Income	Paddy vs Vaikool (Straw) dynamic income form

Current State

OwnFarmIncome.jsx stores harvestType as free-text (e.g. 'Paddy') alongside numberOfBags and pricePerBag. There is no Paddy Straw (Vaikool) option. The unit system is fixed to 'bags' even though paddy straw is counted in bundles (kattu). This does not match real agricultural recording practice.

New Schema

```
{
  id:             'OFI001',
  date:           '2026-04-20',
  incomeSource:   'paddy',          // 'paddy' | 'vaikool'
  // Used when incomeSource = 'paddy':
  numberOfBags:   50,               // மூட்டைகளின் எண்ணிக்கை
  pricePerBag:    2200,             // ஒரு மூட்டை விலை (₹)
  // Used when incomeSource = 'vaikool':
  numberOfBundles: 0,              // கட்டுகளின் எண்ணிக்கை
  pricePerBundle:  0,              // ஒரு கட்டு விலை (₹)
  // Always present (computed on save):
  totalIncome:    110000,
  description:    '',              // free-text description field kept
  source:         'own_farm'
}
```

Dynamic Form Behaviour

- Source dropdown (first field): Paddy (நெல்) | Paddy Straw / Vaikool (வைக்கோல்)
- When 'paddy' selected → show: No. of Bags (மூட்டைகள்), Price per Bag (ஒரு மூட்டை விலை)
- When 'vaikool' selected → show: No. of Bundles (கட்டுகள்), Price per Bundle (ஒரு கட்டு விலை)
- totalIncome auto-computes in both cases: quantity × price (live display as user types)
- Description field remains below totalIncome in both modes

List View Changes

- Show incomeSource label in Tamil: நெல் (for paddy) or வைக்கோல் (for vaikool)
- Show correct unit: 50 மூட்டைகள் (for paddy) or 120 கட்டுகள் (for vaikool)
- Existing paddy records with old schema (harvestType field): treat numberOfBags and pricePerBag as-is, show as நெல்

4. Drivers Module — Full Restructure (C08)

C08	Drivers	Two sub-modules + salary logic fully decoupled from driver logs

Structure Change

Drivers.jsx becomes a tabbed module with two sub-modules. The standalone SalaryDriver.jsx route is removed. The App.jsx /salary-driver route is removed. The driver nav now shows 'My Salary' which renders Drivers.jsx with the salary tab pre-selected and restricted to the logged-in driver's userId.

Tab	Tamil Name	Who Can Access	Contents
Tab 1: Drivers	ஓட்டுநர்கள்	Admin only	Existing add/edit/view/delete driver logic — unchanged
Tab 2: Salary	சம்பளம்	Admin (full) + Driver (read-only own records)	New salary logic — see below

Current Salary Logic — The Problem

The current SalaryDriver.jsx scans for driver logs without a linkedSalaryId. If availableLogs.length === 0, it immediately blocks with an alert and prevents saving. This creates a hard dependency: the salary module cannot record ANY payment unless there are unpaid driver logs available. Real scenarios this breaks:

- Admin wants to give a Diwali bonus: zero logs unpaid → blocked
- Admin wants to record a ₹500 advance before any work starts → blocked
- Admin wants to note extra pay for a driver who helped with non-machine work → blocked
- Driver worked but log entry was not created yet → blocked

New Salary Logic — Fully Independent Entries

Each salary entry is a standalone financial record. There is no dependency on driver logs. Admin can create a salary entry for any combination of base, bonus, extra, and advance at any time.

```
// New salary entry schema (saved to rl_driver_salary):
{
  id:           'DSL001',
  driverId:     'D001',
  date:         '2026-04-20',
  baseSalary:   1200,    // 0 if not this entry type
  bonus:        0,
  extraAmount:  0,
  advance:      0,
```

```
  // netPay is computed on render, NOT stored:
  // netPay = (baseSalary + bonus + extraAmount) - advance
  notes:        '',
  payments:     []       // partial payment tracking (same pattern as Harvester)
}
```

Form Validation Rule

At least one of baseSalary, bonus, extraAmount, or advance must be > 0. If all four are 0 and Save is tapped, show inline Tamil error: 'குறைந்தது ஒரு தொகை தேவை (At least one amount is required)'. All four can be filled simultaneously in one entry — this is the normal combined salary entry. Any single field can be filled alone — this allows bonus-only, advance-only, or extra-only entries.

Net Pay Display

Net Pay is shown live as the user types: (baseSalary + bonus + extraAmount) - advance. If the result is negative (advance exceeds total earnings), show: 'முன்பணம் சம்பளத்தை விட அதிகமாக உள்ளது' with a red background. Negative net pay is allowed to save — it represents a carry-forward advance situation.

Payments[] Tracking (Same Pattern as Harvester)

After a salary record is saved, admin can record partial payments against it using the PaymentHistory component (already built and working). This is the same pattern used in Harvester jobs and Rental entries. sumPayments(salary.payments) gives total paid. balance = netPay - sumPayments. Driver can view balance but cannot add payments.

Edit and Delete — Admin Only

Action	Admin	Driver	Rule
View salary records	All drivers	Own records only	Driver sees read-only list filtered by their userId
Edit salary entry	Yes — full edit of all fields	No	Edit updates the record in rl_driver_salary via updateRecord()
Delete salary entry	Yes	No	deleteRecord() removes from rl_driver_salary; show ConfirmDialog first
Add payment to salary	Yes	No	Existing addPayment() function in storage.js used
Edit or delete a payment	No	No	Payments are immutable; add a correction entry instead

Linking to Logs (Optional Context, Not Required)

The old system used linkedSalaryId on driver logs to mark them as processed. This hard dependency is removed. However, the admin may optionally add a note on a salary entry referencing specific log dates (using the notes field). No automated log-linking is implemented in v3.1 — salary entries are fully independent.

App.jsx Route Changes

```
// REMOVE this route:
<Route path='/salary-driver' element={<SalaryDriver />} />

// UPDATE the driver nav link: instead of '/salary-driver'
// Driver user sees: 'My Salary' → navigates to /drivers?tab=salary
// The Drivers.jsx tab parameter detects ?tab=salary on mount
// and pre-selects Tab 2, filters to userId, and sets isReadOnly=true

// OR simpler approach:
// In App.jsx driver nav, render a link to /drivers with salary tab
// In Drivers.jsx useEffect: if user.role === 'driver', auto-select salary tab
//   and set isReadOnly = true, driverFilter = user.id
```

5. Workers Module — Full Restructure (C09 + C12)

C09	Workers	Two sub-modules + multi-worker entry + filters + PDF reports

Structure Change

Workers.jsx becomes a tabbed module with two sub-modules.

Tab	Tamil Name	Contents
Tab 1: Workers	வேலையாட்கள்	Add, edit, view, deactivate workers. Name, phone, village fields.
Tab 2: Work Records	வேலை பதிவு	Work entry form + filtered list + PDF reports

C12 — Multi-Worker Entry (Same Work, Same Day)

This change comes from the real-world pattern where large agricultural tasks (fertilizing, transplanting seedlings — naatru nattal, kalai yeduthal, andai vettuthal) are done by multiple workers simultaneously. Recording them one by one is time-consuming and error-prone.

Current State

Workers.jsx saves one entry per form submission with a workerName string. To record 5 workers doing the same work, admin must fill the form 5 separate times. This is the root pain point.

New Multi-Worker Entry Logic

```
// Form state for multi-worker entry:
const [formData, setFormData] = useState({
  date: today,
  selectedWorkerIds: [],   // ARRAY — multiple workers can be selected
  workType: '',
  baseSalary: 0,
  bonus: 0,
  extraAmount: 0,
  advance: 0,
  notes: ''
});

// Worker selection UI:
// Show all active workers as a checklist (not a dropdown)
// Each worker: [checkbox] Kumar  [checkbox] Murugan  [checkbox] Ravi
// Admin ticks any combination of workers

// Net Pay display: same formula, computed live
const netPayable = (baseSalary + bonus + extraAmount) - advance;
```

```
// On save — create ONE entry per selected worker:
const handleSubmit = () => {
  if (formData.selectedWorkerIds.length === 0) {
    setError('குறைந்தது ஒரு வேலையாள் தேர்வு செய்யவும்');
    return;
  }
  formData.selectedWorkerIds.forEach(workerId => {
    const entry = {
      id: generateId('rl_work_entries'),
      workerId,            // reference — NOT workerName string
      date: formData.date,
      workType: formData.workType,
      baseSalary:   parseFloat(formData.baseSalary)   || 0,
      bonus:        parseFloat(formData.bonus)        || 0,
      extraAmount:  parseFloat(formData.extraAmount)  || 0,
      advance:      parseFloat(formData.advance)      || 0,
      // netPayable NOT stored — computed on render
      notes: formData.notes
    };
    addRecord('rl_work_entries', entry);
  });
  // Show: 'X workers saved successfully'
  setSuccess(`${formData.selectedWorkerIds.length} வேலையாட்கள் சேமிக்கப்பட்டனர்`);
};
```

workerName → workerId Migration

All new entries save workerId (reference to rl_workers). The display always looks up the worker name dynamically from rl_workers using the workerId. For historical entries that have workerName (string) instead of workerId, the list renders the stored workerName directly with a visual indicator ('Legacy entry') so they remain readable.

Work Records Filters

Filter	UI Element	Logic
Worker filter	Dropdown: All Workers / select one	entries.filter(e => filter===‘all’ || e.workerId===selectedId)
Date range	From Date + To Date inputs	entries.filter(e => e.date >= from && e.date <= to)
Work type filter	Dropdown: All Types / select from workTypes.js	entries.filter(e => filter===‘all’ || e.workType===selected)
Combined	All 3 filters applied simultaneously	AND logic — all conditions must match

PDF Reports — 3 Types

Report Type 1 — Individual Worker Report

- Select one worker from dropdown
- Optional: apply date range filter
- 'Download PDF' → POST /api/pdf/worker/individual
- PDF: Worker name, village, phone at top. Table: Date, Work Type, Base, Bonus, Extra, Advance, Net Pay. Footer: Totals row

Report Type 2 — Filtered Report

- Apply any combination of filters (worker, date, work type)
- 'Download Filtered PDF' → POST /api/pdf/worker/filtered
- PDF: Filter criteria shown at top. Table: Worker name, Date, Work Type, Net Pay. Footer: Total workers, Total amount

Report Type 3 — Overall Report

- No filter required
- 'Download Overall PDF' → POST /api/pdf/worker/overall
- PDF: Summary table with one row per worker. Columns: Worker Name, Total Days, Total Earned. Grand total at bottom.

6. System Module — New Combined Navigation (C10)

C10	Navigation	Merge PricingConfig + Settings + Reports into one System module

Why This Change

The current navigation has three separate items: Config, Settings, Reports. These are rarely-used administrative functions that do not belong in the same nav level as operational modules like Harvester and Rental. Grouping them into a single 'System' module reduces nav clutter and groups administrative concerns together.

New File: src/pages/System.jsx

System.jsx is a new page with 3 tabs. Each tab renders the full content of the corresponding existing page — the existing pages are not deleted, they are imported as sub-components inside System.jsx.

Tab	Tamil Name	Content Source	URL (tab param)
Price Settings	விலை அமைப்பு	PricingConfig.jsx content	/system?tab=price
System Settings	கணினி அமைப்பு	Settings.jsx content + Driver PIN from C04	/system?tab=settings
Reports	அறிக்கைகள்	Reports.jsx content + PDF buttons from C05	/system?tab=reports

App.jsx Route Changes

```
// REMOVE these 3 routes and nav links:
// <Route path='/pricing-config' element={<PricingConfig />} />
// <Route path='/settings'       element={<Settings />} />
// <Route path='/reports'        element={<Reports />} />

// ADD this 1 route:
<Route path='/system' element={<System />} />

// Admin nav: remove 3 items, add 1 'System (அமைப்பு)' item
// Default tab when navigating to /system: Price Settings

// Deep links still work via tab param:
// /system?tab=reports → opens Reports tab directly
// Useful for bookmarking or direct navigation from dashboard
```

7. Admin Edit and Delete Driver Log Entries (C11)

C11	Driver Logs	Admin can edit or delete any driver log entry with cascade guard

Why This Is Needed

Drivers enter logs in the field, often on small mobile screens. Mistakes happen — wrong farmer selected, wrong machine type, incorrect time. Currently there is no way to fix a log after it is saved. The admin has no edit or delete capability on driver log entries. This creates permanent incorrect data in the system.

Where This Feature Appears

In DriverEntry.jsx, the 'Today's Logs' section at the bottom of the page already shows saved logs. The admin view of this page should show ALL logs (not just today's), with edit and delete buttons per entry. Drivers see their own logs as before (today's only, no edit/delete).

Edit Logic

- Admin taps 'Edit (திருத்து)' on any log entry
- The log data populates the form at the top of the page (all fields: driver, farmer, machine type, date, sessions, diesel)
- The form shows in 'Edit Mode' with a header: 'Editing Log: LOG001'
- Admin makes corrections and taps 'Update Log (பதிவை புதுப்பி)'
- updateRecord('rl_driver_logs', log.id, updatedLog) — updates in localStorage

Cascade on Edit — Linked Harvester Job

If the log being edited is linked to a Harvester Job (its id appears in any job's linkedLogIds array), the system must recalculate that job's totalHours and derived fields after the edit:

```
// After updating a driver log:
const linkedJob = getData('rl_harvester_jobs').find(j =>
  j.linkedLogIds.includes(updatedLog.id)
);

if (linkedJob) {
  // Recompute totalHours from all linked logs
  const allLinkedLogs = getData('rl_driver_logs')
    .filter(l => linkedJob.linkedLogIds.includes(l.id));
  const newTotalHours = allLinkedLogs.reduce((sum,l)=>sum+l.totalHours,0);
  const newGross = newTotalHours * linkedJob.ratePerHour;
  const newFinal = Math.max(0, newGross - linkedJob.discount);
  const newDieselFromLogs = allLinkedLogs.reduce((sum,l)=>sum+(l.diesel?.total||
0),0);

  updateRecord('rl_harvester_jobs', linkedJob.id, {
    totalHours:     newTotalHours,
```

```
    grossAmount:    newGross,
    finalAmount:    newFinal,
    dieselFromLogs: newDieselFromLogs,
    // totalExpense and netProfit are recomputed: final - (diesel +
additionalDiesel + otherExp)
  });
}
```

Delete Logic

Guard — Cannot Delete if Log is Linked to a Harvester Job with Payments

```
const handleDeleteLog = (logId) => {
  const linkedJob = getData('rl_harvester_jobs').find(j =>
    j.linkedLogIds.includes(logId)
  );

  if (linkedJob) {
    const hasPaid = linkedJob.payments && linkedJob.payments.length > 0;
    if (hasPaid) {
      // BLOCK DELETE — show Tamil error
      setError('இந்த பதிவு ஒரு செலுத்தப்பட்ட அறுவடை வேலையுடன் இணைக்கப்பட்டுள்ளது.');
      setError('(This log is linked to a harvester job that has payments
recorded.)');
      return;
    } else {
      // Warn but allow — show ConfirmDialog with warning
      setConfirmMsg('இந்த பதிவை நீக்குவது அறுவடை வேலையிலிருந்து தொடர்பை அகற்றும்.
தொடரவா?');
      setPendingDeleteId(logId);
      return;
    }
  }

  // No linked job — standard confirm + delete
  setConfirmMsg('இந்த பதிவை நீக்க விரும்புகிறீர்களா?');
  setPendingDeleteId(logId);
};

// On confirm:
const handleConfirmDelete = () => {
  deleteRecord('rl_driver_logs', pendingDeleteId);
  // Also unlink from any job that referenced this log (if no payments):
  const jobs = getData('rl_harvester_jobs');
  jobs.forEach(job => {
    if (job.linkedLogIds.includes(pendingDeleteId)) {
      const newIds = job.linkedLogIds.filter(id => id !== pendingDeleteId);
      updateRecord('rl_harvester_jobs', job.id, { linkedLogIds: newIds });
      // Recompute job totalHours with remaining logs
    }
  });
};
```

Admin Log List View

When the admin visits DriverEntry.jsx, the bottom section shows ALL logs (not just today's). Filters available: Driver filter, Date range, Machine type. Each log card shows Edit and Delete buttons. Drivers see their own logs (today only) with no edit/delete buttons — same as current behaviour.

8. Backend Changes — New PDF Endpoints

8.1 All New Endpoints in server/index.js

Endpoint	Method	Input	Generator Function
POST /api/pdf/driver-salary-statement	POST	{ driver, salaryRecords, fromDate, toDate }	generateDriverSalaryStatement()
POST /api/pdf/worker/individual	POST	{ worker, entries }	generateWorkerIndividualReport()
POST /api/pdf/worker/filtered	POST	{ entries, filters }	generateWorkerFilteredReport()
POST /api/pdf/worker/overall	POST	{ summary }	generateWorkerOverallReport()
POST /api/pdf/report/monthly	POST	{ month, year, incomeRows, expenseRows }	generateMonthlyReport()
POST /api/pdf/report/machine	POST	{ season, year, rows }	generateMachineReport()
POST /api/pdf/report/farmer-dues	POST	{ rows, asOfDate }	generateFarmerDuesReport()
POST /api/pdf/report/driver-payroll	POST	{ month, year, rows }	generateDriverPayrollReport()
POST /api/pdf/report/seasonal	POST	{ season, year, rows }	generateSeasonalReport()

8.2 PDF Layout Standard (All New Documents)

- Header: Document title in Tamil + English, date generated, filter/period info
- Font: NotoSansTamil for all Tamil text, Helvetica for numbers and English
- Table structure: Navy header row, alternating white/light-gray data rows
- Footer: Page number, company name 'SmartUzhavan — கிராம கணக்கு'
- Totals row: Bold, teal background, at the bottom of every table
- All monetary values: formatted as ₹ X,XXX (using existing formatCurrency logic)

9. Implementation Build Order

Follow this exact order. Each step has dependencies on the previous.
Complete and test each step before moving to the next.
Do NOT combine multiple steps in one response.

St	Change	File(s)	Dependency	Est.

ep				Time
1	Fix DriverEntry empty session defaults (C02)	DriverEntry.jsx	None — isolated 2-line fix	5 min
2	Fix Expenses undefined labels + 3 category maps (C06)	Expenses.jsx	None — isolated data fix	20 min
3	Fix OwnFarmIncome Paddy/Vaikool schema (C07)	OwnFarmIncome.jsx	None — isolated form change	25 min
4	Add Season+Month filter to DriverDashboard (C01)	DriverDashboard.jsx	None — reads existing data	30 min
5	Rebuild Workers.jsx with tabs + workerName→workerId (C09 base)	Workers.jsx	None — self-contained	40 min
6	Add multi-worker checklist entry to Workers.jsx (C12)	Workers.jsx	Step 5 must be done first	20 min
7	Rebuild Drivers.jsx with tabs + new salary logic (C08)	Drivers.jsx + App.jsx	Steps 5,6 done (pattern reference)	60 min
8	Add date filter + PDF button to Salary tab (C03)	Drivers.jsx + server	Step 7 must be done first	30 min
9	Add Driver PIN change to future System module (C04)	Drivers.jsx temp location	Step 7 must be done first	15 min
10	Add Admin log edit/delete with cascade guard (C11)	DriverEntry.jsx + storage	Step 1 must be done first	40 min
11	Create System.jsx — merge Config+Settings+Reports (C10)	System.jsx + App.jsx	Steps 4,7,8,9 done first	30 min
12	Add PDF buttons to Reports tab in System.jsx (C05)	System.jsx + server	Step 11 must be done first	60 min
13	Add 9 new PDF endpoints + generator functions (C03,C05,C09)	server/index.js + pdfGenerator.js	Steps 8,10,12 must be done first	90 min
14	End-to-end test: all roles, all modules, all PDF types	All	All steps complete	30 min

10. Do Not Touch — Working Files

These files are verified working in v3.0. Modifying them without explicit instruction will introduce new

bugs.
If a fix in this plan requires importing from these files, import — do NOT modify.

File	Status
src/services/storage.js	✔️ All 7 functions correct — updateRecord and deleteRecord ready for C11
src/services/calculations.js	✔️ All 9 functions correct
src/services/billId.js	✔️ HB-SSS-YYYY-NNN format correct
src/services/pdfService.js	✔️ Correct (API URL env var fix from v3.0 already applied)
src/utils/timeUtils.js	✔️ Overnight guard correct
src/utils/idGenerator.js	✔️ All prefixes correct
src/data/machineTypes.js, seasons.js, workTypes.js	✔️ All correct
src/pages/Harvester.jsx	✔️ Fully working — do not touch
src/pages/Rental.jsx	✔️ Working — PDF fix from v3.0 applied
src/pages/FarmerView.jsx	✔️ Working
src/pages/Finance.jsx	✔️ Working (lending tab)
src/pages/Reports.jsx	✔️ Working (PDF buttons added in C05 — do not change table logic)
src/pages/Login.jsx	✔️ Working — PIN security fix from v3.0 applied
src/pages/PricingConfig.jsx	✔️ Working — will be imported into System.jsx
src/pages/Settings.jsx	✔️ Working — will be imported into System.jsx
src/components/ (all)	✔️ All components working — reuse PaymentHistory for Salary
server/pdfGenerator.js (existing functions)	✔️ generateHarvesterBill, generateStatement, generateRentalReceipt working

END OF IMPLEMENTATION PLAN — VERSION 3.1

SmartUzhavan — 12 Changes | 8 Modified Files | 3 New Files | 9 New PDF Endpoints